

MACs e Cifras Autenticadas

Q1 — Padding no Length Extension Attack

A questao considera a mensagem URL:

http://www.super-secret.com/manage?id=1001&role=user&perm=read

com extensao &admin=true. O ataque de extensao de comprimento (Length Extension Attack) explora a construcao de Merkle-Damgard usada pelo SHA256, permitindo forjar um MAC valido sem conhecer a chave.

Calculo do Padding

O SHA256 processa a mensagem completa que o hash viu internamente, ou seja:

chave (32 bytes) + mensagem (62 bytes) = 94 bytes no total

O SHA256 processa blocos de 64 bytes. Para completar o bloco, e necessario adicionar padding ate ao multiplo de 64 bytes seguinte (128 bytes = 2 blocos):

$128 - 94 = 34$ bytes de padding necesarios

Estrutura do Padding (RFC 6234)

O padding obtido experimentalmente foi:

[illegible]

Componente	Valor	Tamanho	Significado
Bit marcador	0x80	1 byte	Bit 1 seguido de zeros
Zeros	0x00 x 31	31 bytes	Preenchimento ate ao bloco
Tamanho	0x02F0	2 bytes	Tamanho total em bits: (32+62)x8 = 752 bits
Total		34 bytes	

Verificacao do Ataque

Apos executar o programa `mac_sha256_attack.py`, a verificacao do MAC para a mensagem estendida com a chave original devolve `True`, confirmando que o ataque foi bem sucedido:

```
$ python3 mac_sha256.py ver url.txt.ext mac.key
True
```

O atacante conseguiu forjar um MAC valido para a mensagem estendida sem conhecer a chave, apenas conhecendo o tamanho da chave (32 bytes) e o MAC original. Isto

demonstra a vulnerabilidade do prefix-MAC quando usado com funcoes de hash da familia SHA-2 baseadas na construcao de Merkle-Damgard.

Esta vulnerabilidade e eliminada pelo HMAC, que aplica o hash duas vezes com a chave de formas diferentes, impedindo que o estado interno seja reutilizado pelo atacante.

Q2 — Diferenca entre pbenc_aes_ctr_hmac.py e pbenc_aes_gcm.py

Embora ambos os programas usem internamente o AES no modo CTR para cifrar, os ficheiros produzidos tem tamanhos diferentes. O teste foi realizado com uma mensagem de 33 bytes:

Ficheiro	Tamanho	Composicao
msg_compare.txt	33 bytes	Mensagem original
msg_compare_hmac.enc	97 bytes	salt(16) + nonce(16) + cripto(33) + tag(32)
msg_compare_gcm.enc	77 bytes	salt(16) + nonce(12) + cripto(33) + tag(16)

Diferenca de 20 bytes

A diferenca de 20 bytes entre os dois ficheiros deve-se a dois fatores:

1. Tamanho do NONCE:

- AES-CTR + HMAC usa nonce de 16 bytes (escolha de implementacao)
- AES-GCM usa nonce de 12 bytes (padrao definido para o modo GCM)
- Diferenca: 4 bytes

2. Tamanho da tag de autenticacao:

- HMAC-SHA256 produz uma tag de 32 bytes (256 bits — tamanho do hash SHA256)
- AES-GCM produz uma tag de 16 bytes (128 bits — tamanho do bloco AES)
- Diferenca: 16 bytes

Total: $4 + 16 = 20$ bytes de diferenca, confirmado experimentalmente.

Ausencia de Padding

Em ambos os casos, o criptograma tem exatamente o mesmo tamanho que o texto-limpo (33 bytes). Isto deve-se ao facto de o AES no modo CTR ser uma cifra sequencial — transforma o AES numa cifra de fluxo por meio de um contador, nao sendo necessario adicionar padding para completar blocos, ao contrario do modo CBC.

Comparacao das Abordagens

Característica	AES-CTR + HMAC	AES-GCM
Cifra	AES-CTR	AES-CTR (interno)
Autenticacao	HMAC-SHA256 (manual)	GHASH (integrado)
Chaves necessarias	2 (cifra + MAC)	1
Tamanho do NONCE	16 bytes	12 bytes
Tamanho da tag	32 bytes	16 bytes
Overhead total	48 bytes	28 bytes
Risco de implementacao	Alto	Baixo

O AES-GCM é preferível em implementações modernas por ser mais compacto, mais eficiente e menos sujeito a erros de implementação, uma vez que combina cifra e autenticação numa única operação integrada (AEAD).